

BVCS — A Bash Version Control System

Offline 1 | Bash Scripting

Contents

1	Overview	2
2	End-to-End Example	3
3	Repository Structure: How BVCS works	4
3.1	File Formats	4
3.2	Commit IDs	4
4	Subcommand Reference	5
4.1	init	5
4.2	add	5
4.3	status	5
4.4	commit	6
4.5	log	6
4.6	diff	7
4.7	restore	7
4.8	help	8
5	Tasks	8
5.1	Task 1: Scaffolding & Repository Initialization	8
5.2	Task 2: Staging Files	8
5.3	Task 3: Checking Status	9
5.4	Task 4: Making a Commit	10
5.5	Task 5: Viewing History	11
5.6	Task 6: Viewing Differences	12
5.7	Task 7: Restoring Files	13
6	Useful Commands & Patterns	15
7	Marks Distribution	16
8	Submission Guideline	16

Overview

Motivation. If you have ever accidentally deleted a piece of code you needed, or wanted to see exactly what changed between two versions of a file, you already understand why version control exists. Tools like Git solve this by recording *snapshots* of your project at points you choose, so that you can always look back, compare, and undo.

In this assignment you will build **BVCS** (Bash Version Control System), a simplified but fully functional VCS written entirely in Bash. Every subcommand you implement mirrors a real Git workflow: staging changes, committing snapshots, inspecting history, diffing versions, and restoring files. By the end you will have a working tool you can actually run on a real project and a much clearer mental model of what happens inside a VCS.

Tasks and checkpoints. The assignment is broken into **seven tasks** — one per subcommand (or group of related subcommands). Complete them in order; each task builds on the previous ones. After finishing each task, run the provided **Checkpoint** commands in your terminal. The expected output is shown exactly as it should appear; any difference means something needs fixing before you move on.

Running your script. Rename your submission to `2205XYZ.sh` (replace `XYZ` with your roll-number suffix), make it executable, and set an alias so you can call it as `bvcs`:

```
$ alias bvcs=/path/to/2205XYZ.sh
$ cd /path/to/your/project
$ bvcs <subcommand> [arguments...]
```

Constraint: Use only standard POSIX/GNU utilities available on a typical Linux system (`bash`, `find`, `diff`, `grep`, `wc`, `date`, etc.). Do *not* invoke `git` or any other external VCS tool anywhere in your script.

End-to-End Example

The following session shows what a complete BVCS workflow looks like. Use this as a reference while building.

```
$ alias bvcs=/path/to/bvcs.sh
$ cd /tmp
$ mkdir myproject && cd myproject
$ bvcs init
Initialized empty BVCS repository.

$ echo '#include <stdio.h>' > main.c
$ echo 'int helper() {}' > util.c
$ echo '# Notes' > notes.md

$ bvcs add main.c util.c
Staged: main.c
Staged: util.c

$ bvcs status
Staged for commit:
  main.c
  util.c

Untracked files:
  notes.md

$ bvcs commit -m "Initial commit"
[0001] Initial commit
2 file(s) committed.

$ echo 'int main() {}' >> main.c

$ bvcs status
Modified (not staged):
  main.c

Untracked files:
  notes.md

$ bvcs diff main.c
--- .bvcs/objects/0001/files/main.c
+++ main.c
@@ -1 +1,2 @@
  #include <stdio.h>
+int main() {}

$ bvcs restore main.c
Restore 'main.c' from commit 0001? [y/N]: y
Restored: main.c
```

Repository Structure: How BVCS works

When initialized, BVCS creates a hidden directory `.bvcs/` in the current directory. You must use *exactly* the following layout — your script will read and write these paths directly.

```
.bvcs/
  objects/                # one subdirectory per commit
    0001/
      files/              # snapshot of every committed file
      message             # commit message (plain text, one line)
      timestamp           # commit time (plain text, one line)
    0002/
    ...
  staging                 # staged file paths, one per line
  log                     # commit history, one entry per line
  HEAD                    # commit ID of the most recent commit
```

File Formats

.bvcs/staging — one relative file path per line, no duplicates:

```
src/main.c
include/util.h
```

.bvcs/log — one line per commit, pipe-delimited `id|timestamp|message`:

```
0001|2026-06-26 10:00:00|Initial commit
0002|2026-06-26 11:30:00|Add utility functions
```

.bvcs/HEAD — the commit ID of the latest commit (empty if no commits yet):

```
0002
```

.bvcs/objects/<id>/files/ — a *complete* snapshot of every tracked file at the time of the commit, preserving subdirectory structure. Each commit carries forward all files from the previous commit's snapshot and then overwrites with the newly staged files. This means every snapshot is self-contained: `diff`, `status`, and `restore` always compare against `HEAD` without needing to walk back through history. For example, if `src/main.c` was first committed in 0001 and not staged again in 0002, it still appears in `.bvcs/objects/0002/files/src/main.c`.

.bvcs/objects/<id>/message — the commit message as a single plain-text line.

.bvcs/objects/<id>/timestamp — the commit timestamp as a single plain-text line in the format `YYYY-MM-DD HH:MM:SS`.

Commit IDs

Commit IDs are sequential 4-digit zero-padded integers: 0001, 0002, ...

The ID of the next commit equals the current number of lines in `.bvcs/log` plus one

Subcommand Reference

Subcommand	Description
<code>init</code>	Initialize a new BVCS repository
<code>add <file>...</code>	Stage one or more files for the next commit
<code>status</code>	Show staged, modified, and untracked files
<code>commit -m <msg></code>	Save a snapshot of all staged files
<code>log</code>	Display the full commit history
<code>diff [file]</code>	Compare working copy to the latest commit
<code>restore <file></code>	Restore a file from the latest commit
<code>help</code>	Print usage information for all subcommands

`init`

Creates the `.bvcs/` directory structure in the current directory. All files (`staging`, `log`, `HEAD`) start empty.

```
$ bvcs init
Initialized empty BVCS repository.
```

Error: if `.bvcs/` already exists:

```
Error: BVCS repository already exists.
```

`add`

Adds one or more files to the staging area. Paths must be relative to the repository root.

```
$ bvcs add main.c util.h
Staged: main.c
Staged: util.h
```

Errors:

- No file arguments given — **Error:** No files specified.
- File does not exist — **Error:** 'ghost.txt' not found. (skip and continue with remaining files)
- File already staged — **Already staged:** main.c (skip and continue)

`status`

Displays the state of the working directory.

```
$ bvcs status
Staged for commit:
  main.c
  util.h

Modified (not staged):
  readme.txt

Untracked files:
  build/output.o
```

```
notes.md
```

- **Staged** — files listed in `.bvcs/staging`.
- **Modified** — files present in the HEAD snapshot that are *not* currently staged and whose working-copy content differs from the snapshot.
- **Untracked** — files in the working directory that are neither staged nor present in the HEAD snapshot.

Omit any category that has no files. If all three categories are empty, print:

```
Nothing to commit, working tree clean.
```

Always exclude the `.bvcs/` tree when scanning the working directory.

Ordering and formatting: Staged files appear in the order they were staged (i.e., the order of lines in `.bvcs/staging`). Modified and untracked files are listed in alphabetical order (sorted by path). Each non-empty category is followed by exactly one blank line, including the last category printed.

commit

Records a snapshot of all staged files.

```
$ bvcs commit -m "Add utility functions"
[0002] Add utility functions
2 file(s) committed.
```

Behaviour (in order):

1. Parse the `-m` flag; the commit message is the following argument.
2. Verify staging area is non-empty and a message was provided.
3. Generate the new commit ID.
4. Create `.bvcs/objects/<id>/files/`.
5. If a previous commit exists, copy its entire `files/` tree into the new snapshot (use `cp -r`).
6. For each staged file: create parent directories inside the snapshot, then copy the file, overwriting any carried-forward version.
7. Write `message` and `timestamp` files inside the commit object directory.
8. Append a line to `.bvcs/log`.
9. Overwrite `.bvcs/HEAD` with the new commit ID.
10. Clear `.bvcs/staging` (truncate to empty).

Errors:

- Staging area is empty — **Error:** `Nothing to commit.`
- No message provided — **Error:** `Commit message required. Use -m "message".`

log

Displays commit history in reverse chronological order (most recent first).

```
$ bvcs log
commit 0002
Date:      2026-06-26 11:30:00
```

```
Message:  Add utility functions

commit 0001
Date:     2026-06-26 10:00:00
Message:  Initial commit
```

Print a blank line after each entry, including the last. If no commits exist, print:

```
No commits yet.
```

diff

Shows differences between the working copy and the HEAD snapshot.

- With a file argument: show the diff for that file only.
- Without a file argument: show the diff for every tracked file. Files with no differences display the “no changes” message; they are not omitted.

The diff output must use unified format (`diff -u`) with `-label` flags so that the `--/+++` header lines show the snapshot path and the working-copy path *without* timestamps:

```
$ bvcs diff main.c
--- .bvcs/objects/0002/files/main.c
+++ main.c
@@ -1,4 +1,6 @@
 #include <stdio.h>
+#include <stdlib.h>
...
```

Errors:

- No commits exist — Error: No commits yet.
- File given but not tracked — Error: 'main.c' is not tracked.

If a file has no differences, print `main.c: no changes`.

restore

Restores a file from the HEAD snapshot, overwriting the working copy.

```
$ bvcs restore main.c
Restore 'main.c' from commit 0002? [y/N]: y
Restored: main.c
```

If the user enters anything other than `y` or `Y`, print `Aborted.` and exit with code 0.

Errors:

- No file argument given — Error: No file specified.
- No commits exist — Error: No commits yet.
- File not in HEAD snapshot — Error: 'main.c' not found in commit 0002.

help

Prints a concise usage message listing all subcommands and their syntax. You decide the exact format.

Tasks

Complete the tasks in order. At the end of each task, run the checkpoint to verify your output. Your script must always be the single file `2205XYZ.sh`.

Each checkpoint is a self-contained recipe you can paste directly into your terminal. Set the alias once in your current shell session — replace the path with the actual location of your script:

```
$ alias bvcs=/path/to/your/2205XYZ.sh
```

Every recipe starts by creating a fresh temporary directory so that state from a previous checkpoint does not interfere. Timestamps in `.bvcs/log` and `bvcs log` output reflect the actual time of each commit; the values shown in the examples are illustrative.

Task 1: Scaffolding & Repository Initialization

Goal: Set up the script structure and implement `init`.

1. Create `bvcs.sh` with a correct shebang line (`#!/usr/bin/bash`) and make it executable.
2. Write a `usage` function that prints a brief help message for every subcommand.
3. Write a `check_repo` function that verifies `.bvcs/` exists in the current directory. If it does not, print an error and exit with code 1:

```
Error: Not a BVCS repository. Run 'init' first.
```

4. Write an `init_repo` function that creates the full directory structure described in Section 3.
5. Add the main subcommand dispatcher. Every subcommand *except* `init` and `help` must call `check_repo` before doing any work. For unrecognized subcommands, print `Error: Unknown subcommand '<name>'.` and exit with code 1.

Checkpoint:

```
$ mkdir /tmp/bvcs_task1 && cd /tmp/bvcs_task1
$ bvcs init
Initialized empty BVCS repository.
$ ls -a .bvcs/
.  ..  HEAD  log  objects  staging
$ bvcs init
Error: BVCS repository already exists.
$ (cd /tmp && bvcs status)
Error: Not a BVCS repository. Run 'init' first.
$ bvcs banana
Error: Unknown subcommand 'banana'.
```

Task 2: Staging Files

Goal: Implement `add`.

1. Write an `add_files` function.
2. Error immediately if no arguments were passed.
3. For each file:
 - Check whether the file exists. If not, print an error and move on to the next file.
 - Check whether the file is already staged. If it is, print a notice and `continue`.
 - Otherwise, append the path to `.bvcs/staging` and print a confirmation.

Checkpoint:

```
$ mkdir /tmp/bvcs_task2 && cd /tmp/bvcs_task2
$ bvcs init
Initialized empty BVCS repository.
$ bvcs add
Error: No files specified.
$ echo "hello" > greet.txt && echo "world" > world.txt
$ bvcs add greet.txt world.txt
Staged: greet.txt
Staged: world.txt
$ bvcs add greet.txt
Already staged: greet.txt
$ bvcs add ghost.txt
Error: 'ghost.txt' not found.
$ cat .bvcs/staging
greet.txt
world.txt
```

Task 3: Checking Status

Goal: Implement `status`.

1. Write a `show_status` function. Use an associative array to track which files belong in which category: `staged`, `modified`, or `untracked`.
2. **Staged:** collect all lines from `.bvcs/staging` into an array.
3. **Modified:** (only applicable when HEAD is non-empty.) Read the HEAD commit ID, then iterate over every file stored in `.bvcs/objects/<head>/files/`. Strip the snapshot prefix to recover the relative path. For each such file that is *not* currently staged, compare the working copy to the snapshot. Collect any that differ.
4. **Untracked:** List all working-directory files, excluding `./bvcs/`. Subtract the staged files and the files present in the HEAD snapshot. What remains is untracked.
5. Print each non-empty category in the order shown in the spec. If all three are empty, print the “clean” message instead.

Checkpoint:

```
$ mkdir /tmp/bvcs_task3 && cd /tmp/bvcs_task3
$ bvcs init
Initialized empty BVCS repository.
$ bvcs status
Nothing to commit, working tree clean.
$ echo "hello" > greet.txt && echo "world" > world.txt
$ bvcs add greet.txt world.txt
Staged: greet.txt
```

```
Staged: world.txt
$ echo "# Notes" > notes.md
$ bvcs status
Staged for commit:
  greet.txt
  world.txt

Untracked files:
  notes.md
```

The **Modified** category only appears when a HEAD snapshot exists. Manually build one to verify all three categories at once (this also confirms your implementation reads the object store correctly):

```
$ mkdir -p .bvcs/objects/0001/files
$ cp greet.txt world.txt .bvcs/objects/0001/files/
$ echo "0001" > .bvcs/HEAD
$ > .bvcs/staging
$ echo "goodbye" >> greet.txt
$ bvcs add world.txt
Staged: world.txt
$ bvcs status
Staged for commit:
  world.txt

Modified (not staged):
  greet.txt

Untracked files:
  notes.md
```

Task 4: Making a Commit

Goal: Implement `commit`.

1. Write a `do_commit` function.
2. Parse inputs to extract the `-m` flag and the message string. Error if either is missing.
3. Check that `.bvcs/staging` is non-empty. Error if empty.
4. Generate the commit ID.
5. Create `.bvcs/objects/<id>/files/`.
6. If a previous commit exists (i.e. `HEAD` is non-empty), copy its entire `files/` tree into the new snapshot. This carry-forward ensures every snapshot is complete, so `diff`, `status`, and `restore` can always compare against `HEAD` without walking back through history.
7. Read `.bvcs/staging` line by line. For each file path:
 - Compute the destination path inside the snapshot.
 - Create any required parent directories.
 - Copy the file overwriting any carried-forward version.
8. Write the message to `.bvcs/objects/<id>/message` and the current timestamp to `.bvcs/objects/<id>/timestamp`.

9. Append one pipe-delimited line to `.bvcs/log`.
10. Overwrite `.bvcs/HEAD` with the new commit ID.
11. Truncate `.bvcs/staging` to empty.
12. Print the summary line.

Checkpoint:

```
$ mkdir /tmp/bvcs_task4 && cd /tmp/bvcs_task4
$ bvcs init
Initialized empty BVCS repository.
$ echo "hello" > greet.txt && echo "world" > world.txt
$ bvcs add greet.txt world.txt
Staged: greet.txt
Staged: world.txt
$ bvcs commit
Error: Commit message required. Use -m "message".
$ bvcs commit -m "Initial commit"
[0001] Initial commit
2 file(s) committed.
$ cat .bvcs/HEAD
0001
$ cat .bvcs/log
0001|2026-06-26 10:00:00|Initial commit
$ cat .bvcs/staging
$ ls .bvcs/objects/0001/files/
greet.txt  world.txt
$ bvcs status
Nothing to commit, working tree clean.
$ echo "updated" >> greet.txt
$ bvcs status
Modified (not staged):
  greet.txt

$ bvcs commit -m "empty"
Error: Nothing to commit.
```

Note: The timestamp field in `.bvcs/log` will be the actual time of your commit, not the value shown above.

Task 5: Viewing History

Goal: Implement `log`.

1. Write a `show_log` function.
2. If `.bvcs/log` is empty, print `No commits yet.` and return.
3. Otherwise, read `.bvcs/log` in *reverse* order (most recent first).
4. For each line, extract the three fields separately.
5. Print the formatted entry, followed by a blank line.

Checkpoint:

```
$ mkdir /tmp/bvcs_task5 && cd /tmp/bvcs_task5
$ bvcs init
```

```
Initialized empty BVCS repository.
$ bvcs log
No commits yet.
$ echo "hello" > greet.txt && echo "world" > world.txt
$ bvcs add greet.txt world.txt
Staged: greet.txt
Staged: world.txt
$ bvcs commit -m "Initial commit"
[0001] Initial commit
2 file(s) committed.
$ bvcs add world.txt && bvcs commit -m "Second commit"
Staged: world.txt
[0002] Second commit
1 file(s) committed.
$ bvcs log
commit 0002
Date:      2026-06-26 10:05:00
Message:   Second commit

commit 0001
Date:      2026-06-26 10:00:00
Message:   Initial commit
```

Note: Timestamps in the `bvcs log` output will reflect your actual commit times.

Task 6: Viewing Differences

Goal: Implement `diff`.

1. Write a `show_diff` function.
2. If `.bvcs/HEAD` is empty, error: no commits yet.
3. **With a file argument:**
 - Check that the file exists in `.bvcs/objects/<head>/files/`. If not, error: not tracked.
 - Use `diff -q` to test for differences (exit code 0 = identical).
 - If identical, print `<file>: no changes`.
 - Otherwise, run `diff -u` with two `-label` flags: the first label is the snapshot path (`.bvcs/objects/<head>/files/<file>`), the second is the working-copy path (`<file>`). This suppresses timestamps from the `--/+++` header lines.
4. **Without a file argument:**
 - Enumerate all files in `.bvcs/objects/<head>/files/` in sorted order.
 - For each file, apply the same `diff -u -label` comparison and print either the diff output or the “no changes” message. Every tracked file is shown; none are omitted.

Checkpoint:

```
$ mkdir /tmp/bvcs_task6 && cd /tmp/bvcs_task6
$ bvcs init
Initialized empty BVCS repository.
$ bvcs diff
```

```

Error: No commits yet.
$ echo "hello" > greet.txt && echo "world" > world.txt
$ bvcs add greet.txt world.txt
Staged: greet.txt
Staged: world.txt
$ bvcs commit -m "Initial commit"
[0001] Initial commit
2 file(s) committed.
$ bvcs add world.txt && bvcs commit -m "Second commit"
Staged: world.txt
[0002] Second commit
1 file(s) committed.
$ echo "goodbye" >> greet.txt
$ bvcs diff greet.txt
--- .bvcs/objects/0002/files/greet.txt
+++ greet.txt
@@ -1 +1,2 @@
 hello
+goodbye
$ bvcs diff world.txt
world.txt: no changes.
$ bvcs diff ghost.txt
Error: 'ghost.txt' is not tracked.

```

Task 7: Restoring Files

Goal: Implement `restore`.

1. Write a `restore_file` function that takes a file path as its argument.
2. If `.bvcs/HEAD` is empty, error: no commits yet.
3. Check that the file exists under `.bvcs/objects/<head>/files/`. If not, print an error naming the commit ID.
4. Prompt the user for confirmation. Print the commit ID in the prompt.
5. If the user enters `y` or `Y`:
 - Create any required parent directories in the working tree.
 - Copy the snapshot file back.
 - Print the success message.
6. Otherwise, print `Aborted`.

Checkpoint:

```

$ mkdir /tmp/bvcs_task7 && cd /tmp/bvcs_task7
$ bvcs init
Initialized empty BVCS repository.
$ bvcs restore
Error: No file specified.
$ bvcs restore greet.txt
Error: No commits yet.
$ echo "hello" > greet.txt && echo "world" > world.txt
$ bvcs add greet.txt world.txt
Staged: greet.txt

```

```
Staged: world.txt
$ bvcs commit -m "Initial commit"
[0001] Initial commit
2 file(s) committed.
$ bvcs add world.txt && bvcs commit -m "Second commit"
Staged: world.txt
[0002] Second commit
1 file(s) committed.
$ echo "goodbye" >> greet.txt
$ bvcs restore greet.txt
Restore 'greet.txt' from commit 0002? [y/N]: y
Restored: greet.txt
$ cat greet.txt
hello
$ bvcs restore greet.txt
Restore 'greet.txt' from commit 0002? [y/N]: n
Aborted.
$ bvcs restore ghost.txt
Error: 'ghost.txt' not found in commit 0002.
```

Note: `bvcs restore` prompts for confirmation interactively; type `y` or `n` when the prompt appears.

Useful Commands & Patterns

```
# Unified diff between two files. Exit code 0 means identical.
diff -u file1 file2

# Unified diff with custom labels (path-only headers, no
# timestamps)
diff -u --label "label-for-file1" --label "label-for-file2"
    file1 file2

# Suppress output; only the exit code matters.
diff -q f1 f2

# Print a file in reverse line order (opposite of cat)
tac file

# Print the current date/time in given format.
date +%Y-%m-%d %H:%M:%S

# prints 0007
printf '\%04d' 7

# Count lines in a file
n=$(wc -l < .bvcs/log)

# Check if a file is non-empty
[ -s .bvcs/staging ]

# Check if an exact line exists in a file
grep -qx "$filepath" .bvcs/staging

# Split a pipe-delimited line into separate variables
while IFS='|' read -r id ts msg; do
    # use $id, $ts, $msg
done < .bvcs/log

# List working files, excluding .bvcs/
find . -type f ! -path './.bvcs/*'

# Strip a known prefix from a string (parameter expansion)
prefix=".bvcs/objects/0001/files/"
relpath="${fullpath#$prefix}"

# Copy a file and create parent dirs in one go
dest=".bvcs/objects/$id/files/$file"
mkdir -p "$(dirname "$dest")"
cp "$file" "$dest"
```

Look up manual pages (`man <command>`) for full details.

Marks Distribution

#	Task	Marks
1	Scaffolding & Repository Initialization	5
2	Staging Files	10
3	Checking Status	25
4	Making a Commit	30
5	Viewing History	5
6	Viewing Differences	10
7	Restoring Files	10
8	Help subcommand	5
Total		100

Submission Guideline

- Submit **one file** named *<your id>.sh*.
- It must have the correct shebang line.
- The output produced from running any command should match the specification exactly.
- The file structure after running any command should match the specification exactly.
- All seven tasks must be completed for full marks.
- **Deadline: 10 July 2026 11:45 pm**
- **Do not copy from your classmate. Both of you will get -100%**
- **Do not copy code from LLMs or any website. If you cannot explain your code, you will get -100%**